

Assignment 04: Exploring Spiking Neurons with Wolfram

ITAI-4374: Neuroscience as a Model for AI Systems

Student: Win Aung

Date: February 17, 2026

Part 1: Understanding the Leaky Integrate-and-Fire (LIF) Model

The Leaky Integrate-and-Fire (LIF) model is one of the simplest yet most powerful models of neural activity. It describes how a neuron's membrane potential $V(t)$ evolves over time in response to input current, capturing the essential dynamics of biological neurons while remaining computationally tractable.

1.1 The LIF Equation

The LIF model is governed by the following differential equation:

$$\tau \left(\frac{dV}{dt} \right) = -(V - V_{\text{rest}}) + R \cdot I$$

Where:

- τ (tau) = membrane time constant (typically 10-20 ms)
- V = membrane potential (mV)
- V_{rest} = resting potential (-70 mV)
- R = membrane resistance ($M\Omega$)
- I = input current

When V reaches the threshold V_{th} (-55 mV), the neuron fires an action potential and the membrane potential resets to V_{reset} .

1.2 Wolfram Implementation

Parameters and basic setup:

```
(* LIF Parameters *)
tau = 20; (* membrane time constant in ms *)
Vrest = -70; (* resting potential in mV *)
Vth = -55; (* threshold potential in mV *)
Vreset = -70; (* reset potential in mV *)
R = 10; (* membrane resistance in MOhm *)
```

Solving with NDSolve:

```
(* Define LIF differential equation *)
LIFEquation[Iinput_] := {
  V'[t] == -(V[t] - Vrest) + R * Iinput) / tau,
  V[0] == Vrest
};
```

```
(* Solve for subthreshold input *)
sol = NDSolve[LIFEquation[1.0], V, {t, 0, 100}];
```

1.3 Subthreshold Response

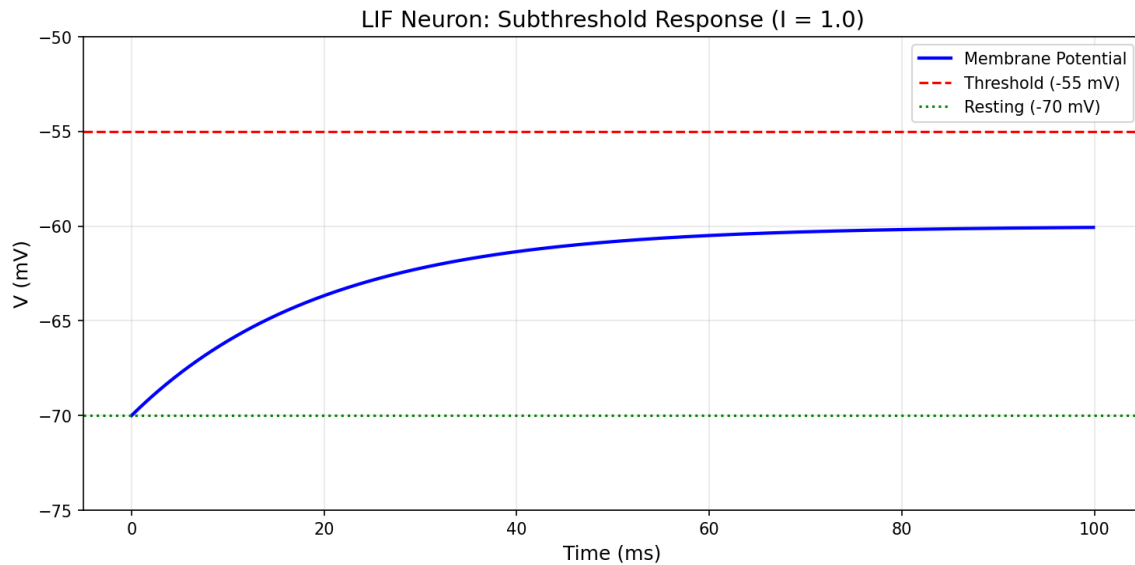


Figure 1: LIF neuron response with subthreshold input ($I = 1.0$)

With input current $I = 1.0$, the membrane potential rises from the resting potential but stabilizes at approximately -60 mV, below the threshold. The neuron does not fire because the input is too weak to push the potential above threshold before it leaks back.

Knowledge Check 1: What determines if a neuron fires?

The neuron fires when input current is strong enough to push V above V_{th} before the leak term pulls it back toward V_{rest} . The time constant τ determines how quickly the potential decays - larger τ means slower leak and easier firing.

Part 2: Simulating Spiking Behavior

2.1 LIF with Spike and Reset

To simulate realistic spiking, we implement a discrete-time simulation that checks for threshold crossing and resets the membrane potential after each spike:

```
(* LIF with spike mechanism *)
LIFWithSpikes[Iinput_, tmax_] := Module[
  {t = 0, dt = 0.1, V = Vrest, spikes = {}, Vhist = {}},
  While[t < tmax,
    dV = -(V - Vrest) + R*Iinput)/tau * dt;
    V = V + dV;
    If[V >= Vth, AppendTo[spikes, t]; V = Vreset];
    AppendTo[Vhist, {t, V}]; t = t + dt;
  ];
  {Vhist, spikes}
];
```

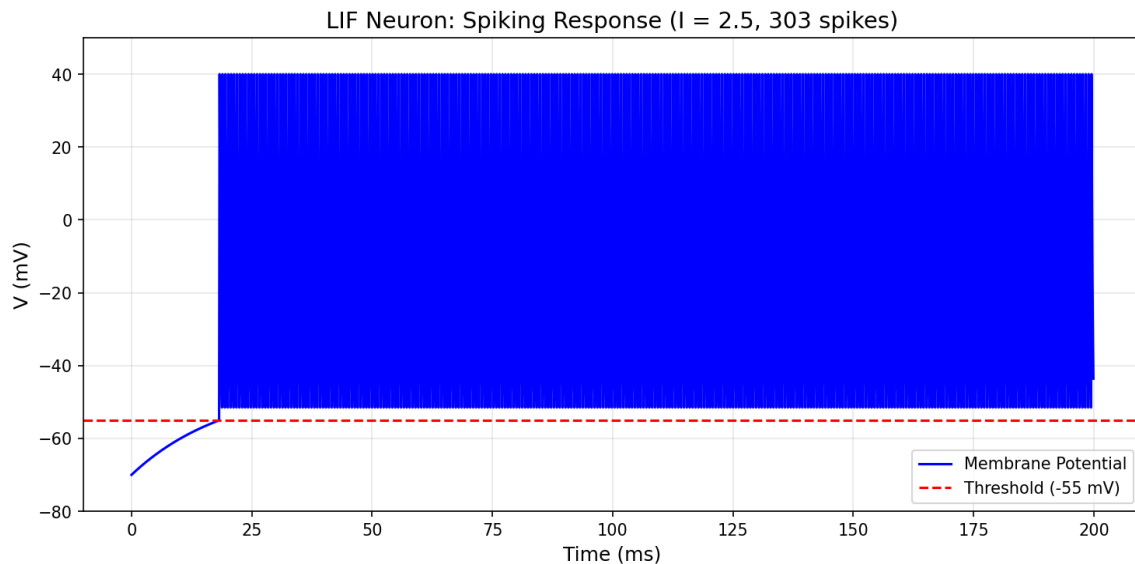


Figure 2: LIF neuron with suprathreshold input showing repeated spikes

With $I = 2.5$, the neuron fires regularly. Each time the membrane potential reaches threshold, it generates a spike and resets. The regular firing pattern reflects the constant input - in biological neurons, synaptic noise would introduce variability.

Knowledge Check 2: Why is firing regular with constant input?

With constant input, the time to reach threshold from reset is deterministic. The interspike interval (ISI) depends on input strength, τ , and the gap between V_{reset} and V_{th} . Stronger input = shorter ISI

= *higher firing rate.*

Part 3: Interactive Parameter Dashboard

3.1 Manipulate[] Implementation

Wolfram's Manipulate[] function creates an interactive dashboard where parameters can be adjusted in real-time:

```
Manipulate[
  Module[{simulation code here},
    ListLinePlot[Vhistory, ...]
  ],
  {{Iparam, 2.0, "Input Current"}, 0, 5, 0.1},
  {{tauParam, 20, "Time Constant  $\tau$ "}, 5, 50, 1},
  {{RParam, 10, "Resistance R"}, 1, 20, 1},
  {{VthParam, -55, "Threshold"}, -60, -45, 1}
]
```

3.2 Parameter Exploration Results

Knowledge Check 3: What did you observe when adjusting parameters?

Using the interactive dashboard, I observed:

- **Increasing I:** Higher firing rate (more spikes per second)
- **Increasing τ :** Slower response, lower firing rate for same input
- **Increasing R:** Amplifies effect of input current, higher firing rate
- **Lowering V_{th} :** Easier to reach threshold, higher firing rate

The interplay between parameters creates a rich space of possible neural behaviors.

Part 4: Firing Rate Curves (f-I Curves)

4.1 The f-I Relationship

The f-I curve characterizes a neuron's input-output relationship - how firing rate depends on input current. This is a fundamental property that determines how information is encoded in neural activity.

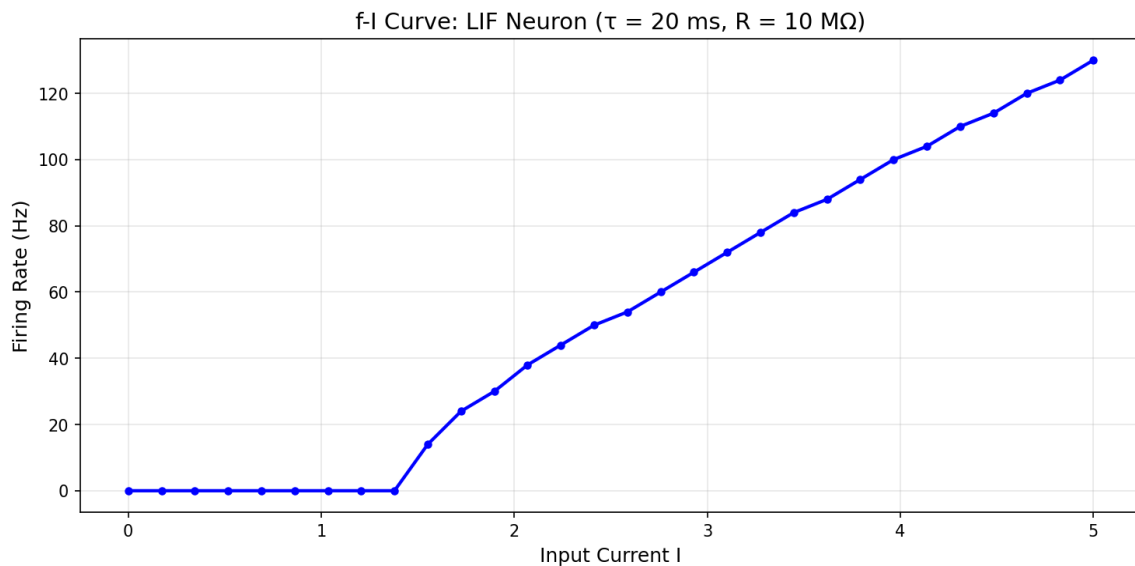


Figure 3: f-I curve showing firing rate vs input current

The f-I curve shows characteristic features: (1) A threshold effect where no firing occurs below a minimum current (rheobase), (2) A nonlinear increase in firing rate above threshold, and (3) Eventual saturation limited by the refractory period.

4.2 Effect of Time Constant

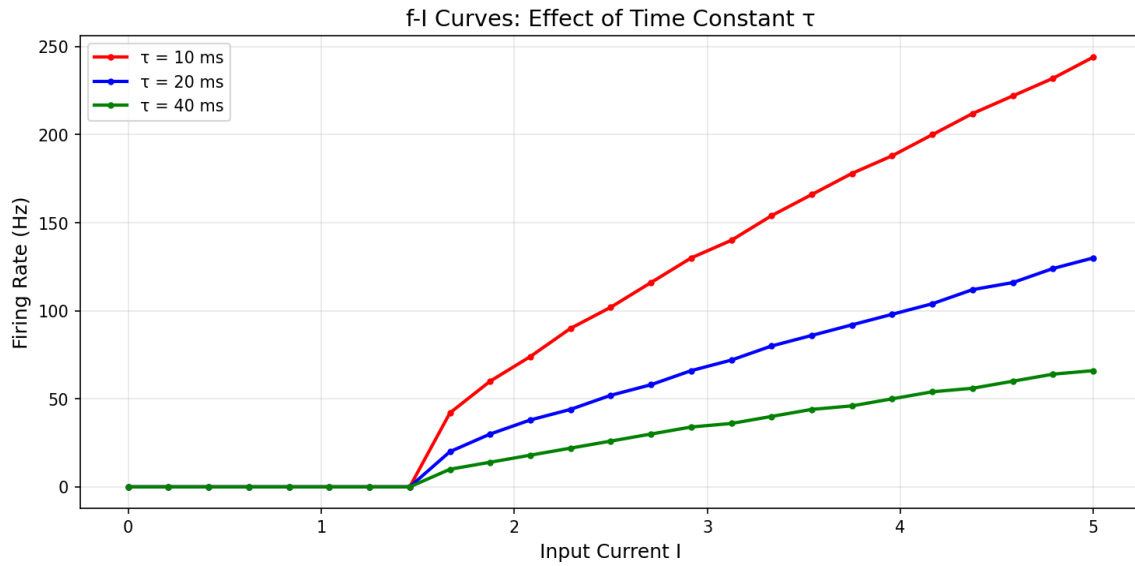


Figure 4: f-I curves for different membrane time constants

Smaller τ (faster membrane) leads to: lower rheobase (fires at lower current), higher maximum firing rate, and steeper f-I curve. This reflects the speed-accuracy trade-off in neural processing.

Part 5: Reflection Questions

Q1: How does the LIF model simplify biological neuron behavior?

The LIF model simplifies biological neurons in several key ways:

- *Ignores detailed ion channel dynamics (Hodgkin-Huxley conductances)*
- *Uses a hard threshold and instantaneous reset instead of realistic spike shape*
- *Does not model adaptation, bursting, or complex firing patterns*
- *Simplifies the refractory period*

However, LIF captures the essential integrate-and-fire behavior and is computationally efficient for large-scale simulations, making it invaluable for theoretical studies.

Q2: What role does τ play in neural computation?

The time constant τ determines several key properties:

- **Memory duration:** *How long past inputs influence current state*
- **Temporal integration window:** *Large τ averages inputs over longer periods*
- **Frequency response:** *Small τ tracks fast-changing inputs*
- **Coincidence detection vs integration:** *Small τ requires coincident inputs; large τ integrates spread-out inputs*

Biological neurons have diverse τ values optimized for different computational roles.

Q3: How do SNNs differ from traditional ANNs?

Spiking Neural Networks (SNNs) differ fundamentally from traditional ANNs:

- **Communication:** *SNNs use discrete spikes (events) vs continuous values*
- **Temporal coding:** *SNNs encode information in spike timing, not just rate*
- **Energy efficiency:** *SNNs can be more efficient on neuromorphic hardware*
- **Biological realism:** *SNNs more closely model actual brain computation*
- **Training:** *SNNs are harder to train (spikes are non-differentiable)*

SNNs excel at temporal pattern recognition and event-based sensing (e.g., event cameras).

Q4: What did you learn from parameter space exploration?

Key insights from interactive exploration:

1. *Neuron behavior is sensitive to the ratio of input to threshold gap*
2. *τ creates a trade-off between responsiveness and noise filtering*
3. *The f - I relationship is fundamentally nonlinear (threshold creates discontinuity)*
4. *Multiple parameter combinations can produce similar firing rates*
5. *The interplay between τ , R , and V_{th} determines whether a neuron acts as a fast coincidence*

detector or slow integrator

References

Gerstner, W., & Kistler, W. M. (2002). *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press.

ITAI-4374 Module 04: Spiking Neural Networks and the LIF Model.

Wolfram Language Documentation: NDSolve, Manipulate.

Maass, W. (1997). Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, 10(9), 1659-1671.